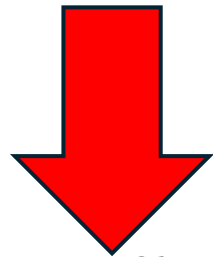


Fat Jar Files

- include **all dependencies within the archive**.
 - self-contained approach
 - simplifies deployment,
 - as the application can be run **without relying on external libraries**.
- Also referred as Uber Jars or executable Jars,



Creating a Fat Jar File:

Using tools like Maven or Gradle, you can generate a Fat Jar easily.

Maven™



- software project management and comprehension tool. Based on the concept of a project object model (POM), Maven can manage a project's build, reporting and documentation from a central piece of information.



<https://maven.apache.org/>

Why Maven in Java?

- Solves the dependency management problem.
- Downloads third-party JAR files that some apps require.
- Provides an extensive, third-party plugin architecture.
- Enables switching between development profiles.
- Integrates with DevOps tools such as Jenkins and Docker.

Apache Maven Tutorial



Last updated: February 25, 2026

 Written by:
baeldung

 Reviewed by:
Kevin Gilmore

Maven

Maven Basics

reference



Code your way through and build up a solid, practical foundation of Java:

[>> Learn Java Basics](#)

1. Introduction

Building a software project typically consists of such tasks as downloading dependencies, putting additional jars on a classpath, compiling source code into binary code, running tests, packaging compiled code into deployable artifacts such as JAR, WAR, an

Apache <https://www.baeldung.com/maven>
building the software manually and separating the work of compiling and packaging our

What is Maven Used For?

- **easily build a project using maven.**
- **add jars and other dependencies of the project easily using the help of maven.**
- Maven is very helpful for a project while **updating the central repository of JARs and other dependencies.**
- With the help of Maven, **we can build any number of projects into output types like the JAR, WAR, etc** without doing any scripting.
- helps in managing the project's build lifecycle, including tasks like **compiling, testing, packaging, and deploying the code.**
- **provides a standard project structure**, making it easy for developers to understand the layout of the project and locate specific files.
- plugins can be used to add **additional functionality to the build process**, such as code coverage analysis, static code analysis, and more.
- simplifies **the process of managing project dependencies**, ensuring that the correct versions of libraries and frameworks are used throughout the project.

Maven java projects have a common structure

```
cd my-app
```

```
my-app
|-- pom.xml
`-- src
    |-- main
    |   |-- java
    |   |   |-- com
    |   |   |   |-- mycompany
    |   |   |   |   |-- app
    |   |   |   |   |   |-- App.java
    |-- test
    |   |-- java
    |   |   |-- com
    |   |   |   |-- mycompany
    |   |   |   |   |-- app
    |   |   |   |   |   |-- AppTest.java
```

src/main/java	Application/Library sources
src/main/resources	Application/Library resources
src/main/filters	Resource filter files
src/main/webapp	Web application sources
src/test/java	Test sources
src/test/resources	Test resources
src/test/filters	Test resource filter files
src/it	Integration Tests (primarily for plugins)
src/assembly	Assembly descriptors
src/site	Site
LICENSE.txt	Project's license
NOTICE.txt	Notices and attributions required by libraries that the project depends on
README.txt	Project's readme

<https://maven.apache.org/guides/introduction/introduction-to-the-standard-directory-layout.html>

Maven java projects have a pom.xml

```
cd my-app
```

```
my-app
|-- pom.xml
`-- src
    |-- main
    |   |-- java
    |   |   |-- com
    |   |   |   |-- mycompany
    |   |   |   |   |-- app
    |   |   |   |   |   |-- App.java
    |-- test
    |   |-- java
    |   |   |-- com
    |   |   |   |-- mycompany
    |   |   |   |   |-- app
    |   |   |   |   |   |-- AppTest.java
    |-- it
    |   |-- java
    |   |   |-- com
    |   |   |   |-- mycompany
    |   |   |   |   |-- app
    |   |   |   |   |   |-- AppIT.java
    |-- assembly
    |   |-- descriptor.xml
    |-- site
    |   |-- site.xml
    |-- LICENSE.txt
    |-- NOTICE.txt
    |-- README.txt
```

src/main/java	Application/Library sources
src/main/resources	Application/Library resources
src/main/filters	Resource filter files
src/main/webapp	Web application sources
src/test/java	Test sources
src/test/resources	Test resources
src/test/filters	Test resource filter files
src/it	Integration Tests (primarily for plugins)
src/assembly	Assembly descriptors
src/site	Site
LICENSE.txt	Project's license
NOTICE.txt	Notices and attributions required by libraries that the project depends on
README.txt	Project's readme

<https://maven.apache.org/guides/introduction/introduction-to-the-standard-directory-layout.html>

You need a POM.xml

- Create it using several options
- Have already one and place it in a folder with the source code

The POM.XML

pom.xml (Project Object Model)

- **Fundamental** file in Maven
- Xml file
- Contains:
 - configuration,
 - dependencies, and
 - project information required for building a Java project.
- details about
 - the project and
 - dependencies (libraries or frameworks),
 - plugins,
 - goals,
 - and build lifecycle.

pom.xml (Project Object Model)

- **Fundamental** file in Maven
- Xml file
- Contains:
 - configuration,
 - dependencies, and
 - project information requi
- details about
 - the project and
 - dependencies (libraries o
 - plugins,
 - goals,
 - and build lifecycle.

Pom.XML are “strange” files

But ...

... you have plenty of examples online

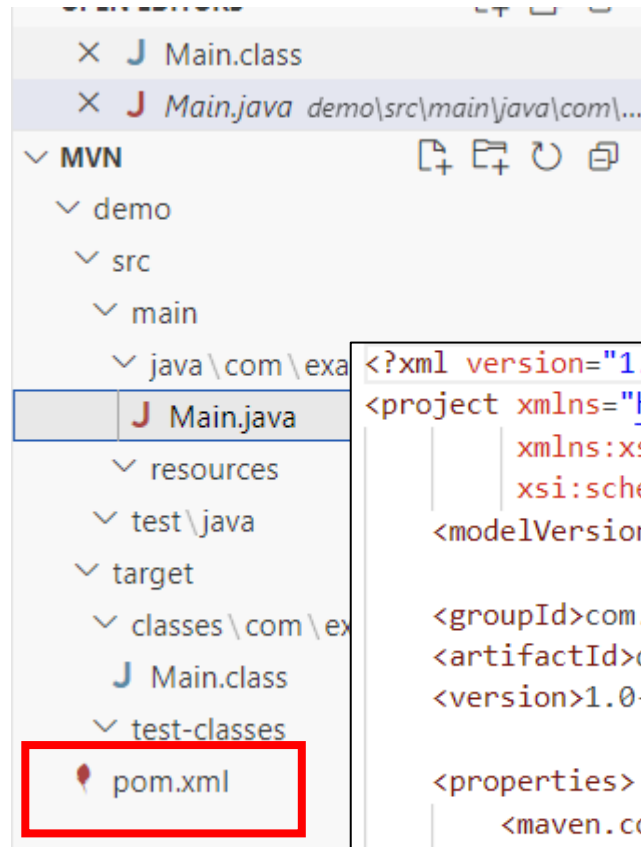
... can ask LLM for custom examples

... you can reuse for older projects

Most of the time, you only need to “cut and paste” specific configurations depending on your project

IDE already provide wizards that put your options in pom.xml

The pom.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>demo</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

</project>
```

EXPLORER

OPEN EDITORS

- Main.class
- pom.xml demo

MVN

- demo
 - src
 - main
 - java\co
 - Main.j
 - resourc
 - test\java
 - target
 - classes\
 - Main.cl
 - test-clas
 - pom.xml

OUTLINE

MAVEN

- demo com.example:demo
 - Lifecycle
 - Plugins
 - Dependencies
 - Favorites
 - Profiles

Your application identification

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>demo</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

</project>
```

Every pom.xml should have

- groupId - the id of the project's group.
- artifactId - the id of the artifact (project)
- version - the version of the artifact under the specified group

EXPLORER

OPEN EDITORS

- Main.class
- pom.xml demo

MVN

- demo
 - src
 - main
 - java\com\example
 - Main.java
 - resources
 - test\java
 - target
 - classes\com\exa
 - Main.class
 - test-classes
 - pom.xml

OUTLINE

MAVEN

- demo com.examp
 - Lifecycle
 - Plugins
 - Dependencies
 - Favorites
 - Profiles

New Maven Project

com.example

Input group Id of your project. (Press 'Enter' to confirm or 'Escape' to cancel)

New Maven Project

demo

Input artifact Id (also as project name) of your project. (Press 'Enter' to confirm or 'Escape' to cancel)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>demo</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

</project>
```

Some details on Pom.xml

The pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.mycompany.app</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <name>my-app</name>
  <!-- FIXME change it to the project's website -->
  <url>http://www.example.com</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.release>17</maven.compiler.release>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.junit</groupId>
        <artifactId>junit-bom</artifactId>
        <version>5.11.0</version>
```

Your application identification

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.mycompany.app</groupId>
```

```
<artifactId>my-app</artifactId>
```

```
<version>1.0-SNAPSHOT</version>
```



The information of project

```
<name>my-app</name>
```



The official application name

```
<!-- FIXME change it to the project's website -->
```

```
<url>http://www.example.com</url>
```

```
<properties>
```

```
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

```
<maven.compiler.release>17</maven.compiler.release>
```

```
</properties>
```



Java environment settings

```
<dependencyManagement>
```

```
<dependencies>
```

```
<dependency>
```

```
<groupId>org.junit</groupId>
```

```
<artifactId>junit-bom</artifactId>
```

```
<version>5.11.0</version>
```

The pom.xml: the environment

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.mycompany.app</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <name>my-app</name>
  <!-- FIXME change it to the project's website -->
  <url>http://www.example.com</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.release>17</maven.compiler.release>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.junit</groupId>
        <artifactId>junit-bom</artifactId>
        <version>5.11.0</version>
```

Java environment settings



The pom.xml: the environment

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.mycompany.app</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <name>my-app</name>
  <!-- FIXME change it to the project's website -->
  <url>http://www.example.com</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.release>17</maven.compiler.release>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.junit</groupId>
        <artifactId>junit-bom</artifactId>
        <version>5.11.0</version>
      </dependency>
    </dependencies>
  </dependencyManagement>
</project>
```

Java environment settings



The pom.xml: dependencies

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <scope>test</scope>
  </dependency>
  <!-- Optionally: parameterized tests support -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-params</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

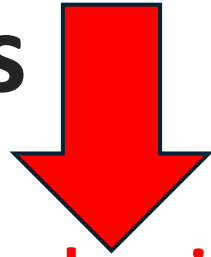
<build>
  <pluginManagement><!-- lock down plugins versions to avoid using Maven defaults (may be moved to parent pom) -->
    ... lots of helpful plugins
  </pluginManagement>
</build>
</project>
```



Depend on specific packages

Projects and dependencies

Fat Jar Files



REMEMBER

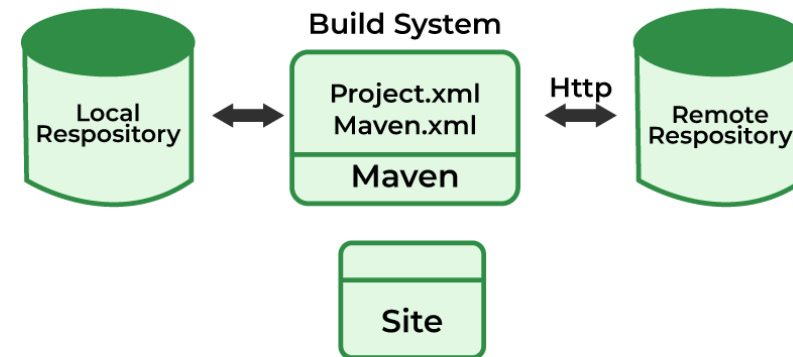
- include **all dependencies within the archive.**
 - self-contained approach
 - simplifies deployment,
 - as the application can be run **without relying on external libraries.**
- Also referred as Uber Jars or executable Jars,

Creating a Fat Jar File:

Using tools like Maven or Gradle, you can generate a Fat Jar easily.

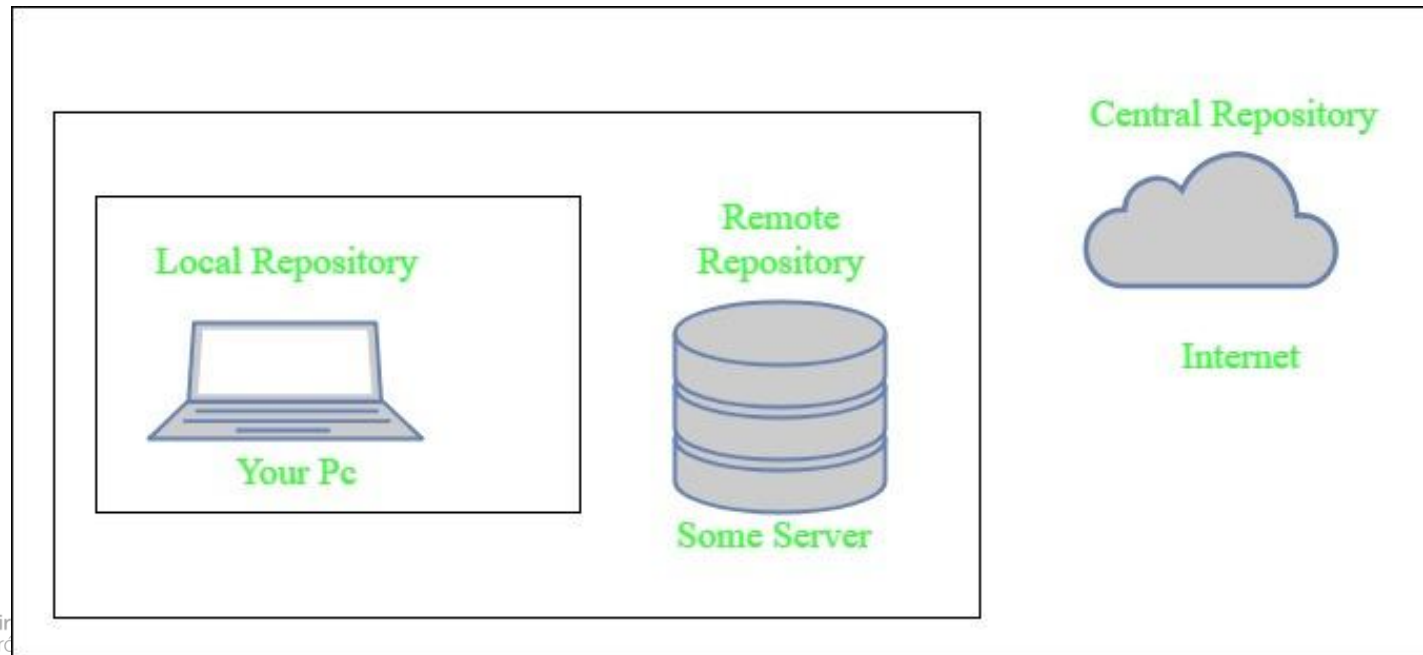
Maven looks for dependencies

- In Repositories = In local repository
- First looks
 - in local Maven repository
 - directory on your machine's hard drive.
 - On central Maven repository
- If successful puts them in your local repository.



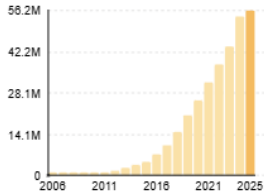
Look for dependencies

- Locally
- Remote repository
- Internet



https://mvnrepository.com/

MVN REPOSITORY

Indexed Artifacts (56.2M)

Popular Categories

- Testing Frameworks & Tools
- Android Packages
- Logging Frameworks
- JVM Languages
- Java Specifications
- JSON Libraries
- Core Utilities
- Mocking
- Web Assets
- Annotation Libraries
- Language Runtime
- HTTP Clients
- Logging Bridges
- Dependency Injection
- XML Processing

What's New in Maven

**Jeddict AI Assistant**
[io.github.jeddict](#) » [jeddict-ai](#) » 2.9 Apache

Jeddict AI Assistant is a powerful and intuitive plugin designed for Apache NetBeans IDE. It integrates AI-powered assistance to help developers work more efficiently with Jakarta EE. This plugin provides real-time code suggestions, documentation insights, and best practices, significantly reducing development time. It is ideal for developers working on complex enterprise applications using Jakarta EE technologies, offering tailored recommendations based on the context ...

Last Release on May 4, 2025

**Markdown**
[io.github.edadma](#) » [markdown](#) » 0.0.7 ISC

markdown

Last Release on May 4, 2025

**Markdown**
[io.github.edadma](#) » [markdown](#) » 0.0.7 ISC

markdown

Last Release on May 4, 2025

**Markdown**
[io.github.edadma](#) » [markdown](#) » 0.0.7 ISC

markdown

Last Release on May 4, 2025

You can find here dependencies by using:

- groupId - the id of the project's group.
- artifactId - the id of the artifact (project)

Search for dependency

The screenshot shows the Maven Repository search results for the term "jackson". The search bar at the top contains "jackson" and the results are sorted by "Relevance". The first result is "Jackson Databind" with 33,255 usages and an Apache license. A blue box highlights the XML dependency snippet:

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-core</artifactId>
  <version>2.19.0</version>
</dependency>
```

Below this, the "Jackson Core" artifact page is shown. A blue arrow points from the "The jar" label to the "jar" file in the "Files" section. The "Files" section shows "pom (9 KB)" and "jar (585 KB)".

On the left side of the screenshot, the "version" dropdown is open, showing a list of versions including 2.19.0, 2.18.2, 2.18.1, 2.18.0, and 2.18.0-rc1. The "2.19.0" version is selected.

At the bottom right, a blue box contains the text "Place in pom.xml" next to the XML dependency snippet.

Annotations on the screenshot include:

- name**: points to the "Jackson Databind" result.
- version**: points to the version dropdown menu.
- The jar**: points to the "jar" file in the "Files" section.
- Place in pom.xml**: points to the XML dependency snippet.

https://central.sonatype.com/



Find OSS Components

As stewards of Central for nearly 20 years and inventors of both software supply chain management and Next Repository, Sonatype knows that the integrity of your build is critical.

José Maria Fernandes (jfernand@ua.pt) is signed in

[Advanced Options](#)

2024 State of the Software Supply Chain

Open source consumption has exploded, with estimates placing **this year's downloads at over 6.6 trillion**. This reliance on open source components, now making up to 90% of the modern software application,



https://central.sonatype.com/

The screenshot shows the Maven Central Repository page for the **jackson-core** artifact. The page has a dark blue header with the "maven central repository" logo and a search bar. Below the header, the artifact name "jackson-core" is displayed, along with a version selector set to "2.19.0" and a badge indicating it is "Used in: 1566 components". The page is divided into tabs: "Overview" (selected), "Versions", "Dependents", and "Dependencies". The "Overview" section contains a "Description" box stating "Core Jackson processing abstractions (aka Streaming API), implementation for JSON" and a "Snippets" section. The "Snippets" section shows a code block for an Apache Maven dependency, with a "Copy to clipboard" button. The code block contains the following XML snippet:

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-core</artifactId>
  <version>2.19.0</version>
</dependency>
```

At the bottom right, there is a "Feedback" button with a question mark icon.

https://central.sonatype.com/

The screenshot shows the Maven Central Repository page for the **jackson-core** artifact. The page has a dark blue header with the "maven central repository" logo and a search bar. Below the header, the artifact name "jackson-core" is displayed, along with its version "2.19.0" and a note that it is "Used in: 1566 components". The page is divided into tabs: "Overview" (selected), "Versions", "Dependents", and "Dependencies". The "Overview" section contains a "Description" box stating "Core Jackson processing abstractions (aka Streaming API), implementation for JSON" and a "Snippets" section. The "Snippets" section shows a code block for an Apache Maven dependency, with a "Copy to clipboard" button. The code block contains the following XML snippet:

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-core</artifactId>
  <version>2.19.0</version>
</dependency>
```

At the bottom right of the page, there is a "Feedback" button with a question mark icon.

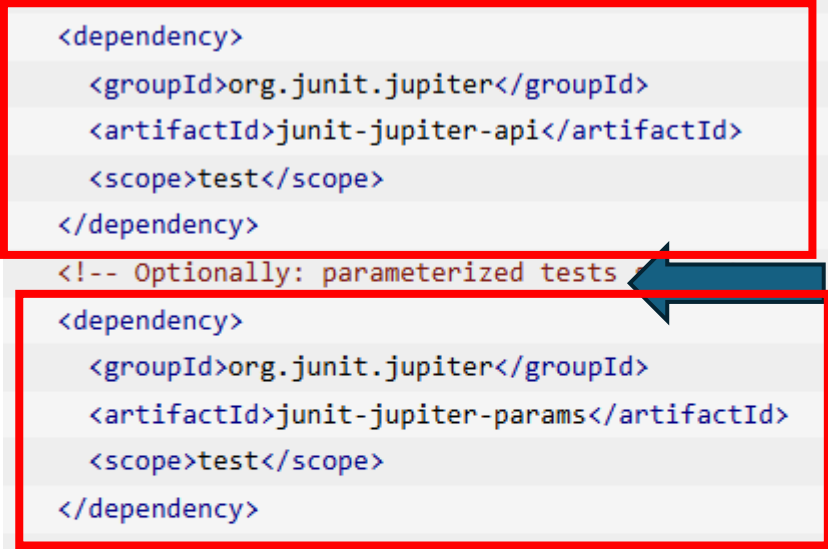
The pom.xml: dependencies

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <scope>test</scope>
  </dependency>
  <!-- Optionally: parameterized tests -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-params</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

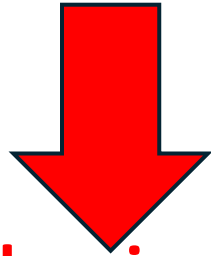
<build>
  <pluginManagement><!-- lock down plugins versions to avoid using Maven defaults (may be moved to parent pom) -->
    ... lots of helpful plugins
  </pluginManagement>
</build>
</project>
```

Depend on specific packages

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-core</artifactId>
  <version>2.19.0</version>
</dependency>
```



Fat Jar Files

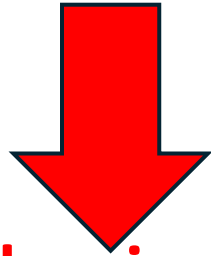


- include **all dependencies within the archive- POM.XML**.
 - self-contained approach
 - simplifies deployment,
 - as the application can be run **without relying on external libraries**.
- Also referred as Uber Jars or executable Jars,

Creating a Fat Jar File:

Using tools like Maven or Gradle, you can generate a Fat Jar easily.

Fat Jar Files



PS. NOT EXACTLY

- include **all dependencies within the archive- POM.XML.**
 - self-contained approach
 - simplifies deployment,
 - as the application can be run **without relying on external libraries.**
- Also referred as Uber Jars or executable Jars,

Creating a Fat Jar File:

Using tools like **Maven** or **Gradle**, you can generate a Fat Jar easily.

Hands on java with maven

Overview

SETUP

GET STARTED

CONFIGURE

EDIT CODE

BUILD, DEBUG, TEST

SOURCE CONTROL

TERMINAL

GITHUB COPILOT

LANGUAGES

NODE.JS /
JAVASCRIPT

TYPESCRIPT

PYTHON

JAVA

Getting Started

Navigate and Edit

Refactoring

Formatting and Linting

Project Management

Build Tools

Java build tools in VS Code

Edit

IN THIS ARTICLE

Maven

Gradle

Additional resources

Next steps

 [RSS Feed](#)

 [Ask questions](#)

 [Follow @code](#)

 [Request features](#)

 [Report issues](#)

 [Watch videos](#)

This document is an overview of how to work with your Java build tools in Visual Studio Code. It covers the [Maven for Java](#) and [Gradle for Java](#) extensions as well as other tools.

If you run into any issues when using the features below, you can contact us by entering an [issue](#).

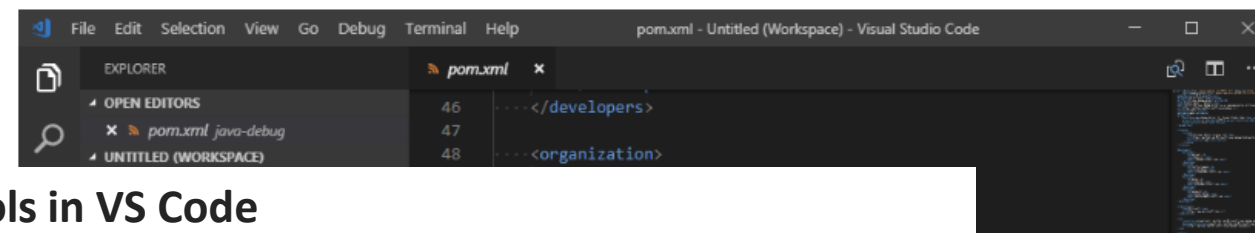
Maven

[Maven](#) is a software tool that helps you manage Java projects and automate application builds. The [Maven for Java](#) extension for Visual Studio Code provides fully integrated Maven support, allowing you to explore Maven projects, execute Maven commands, and perform the goals of build lifecycle and plugins. We recommend installing the [Extension Pack for Java](#), which includes Maven support and other important Java development features.

Install the Extension Pack for Java

Exploring Maven project

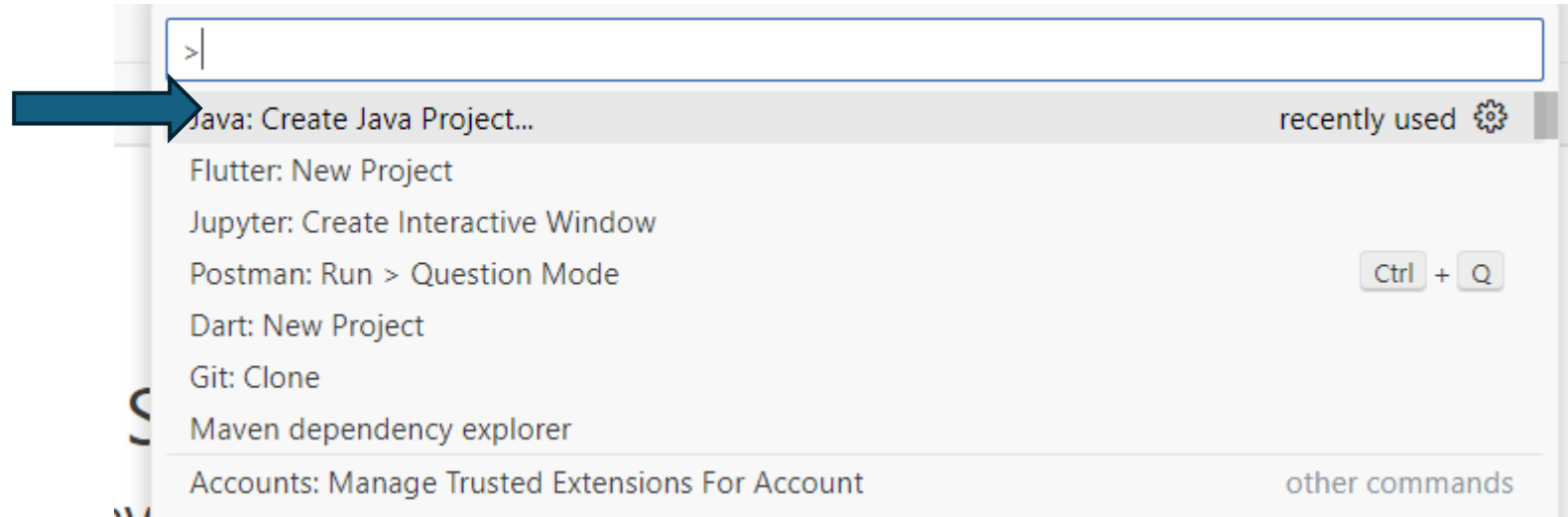
Once a Maven project is loaded, the extension will be activated and it will automatically scan for `pom.xml` files in your workspace and displays all Maven projects and their modules in the side bar.



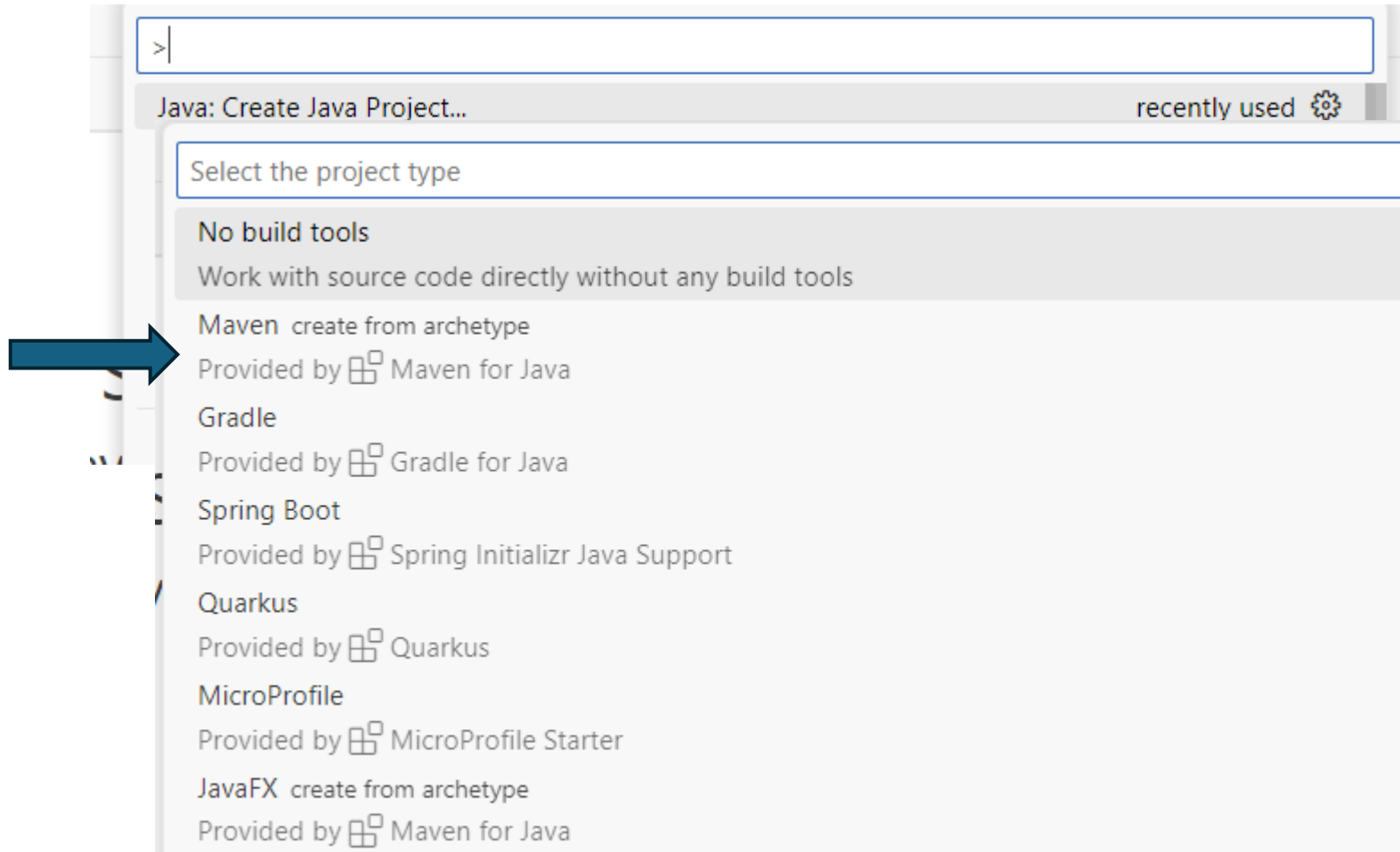
Java build tools in VS Code

<https://code.visualstudio.com/docs/java/java-build>

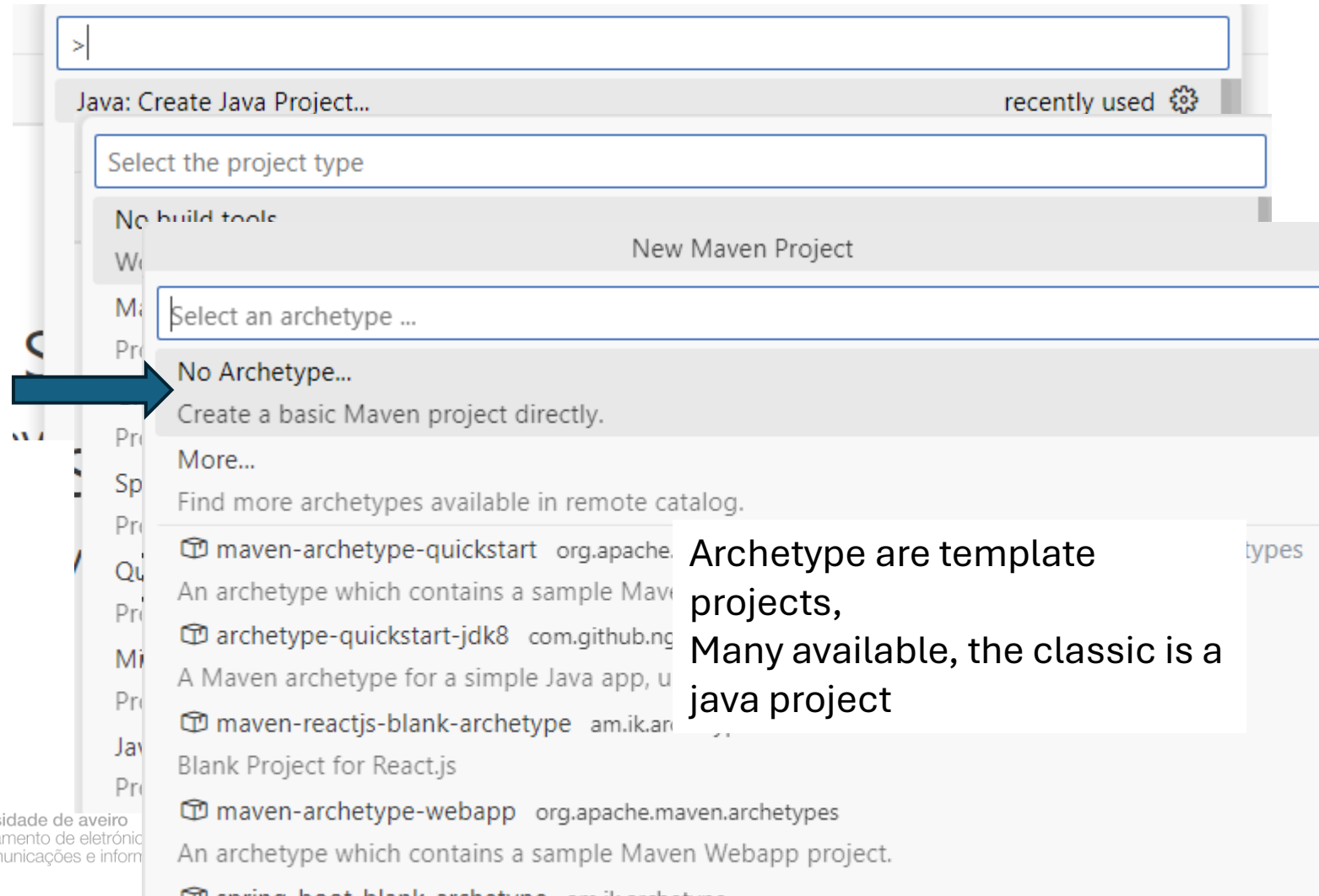
Create a project (VSCode)

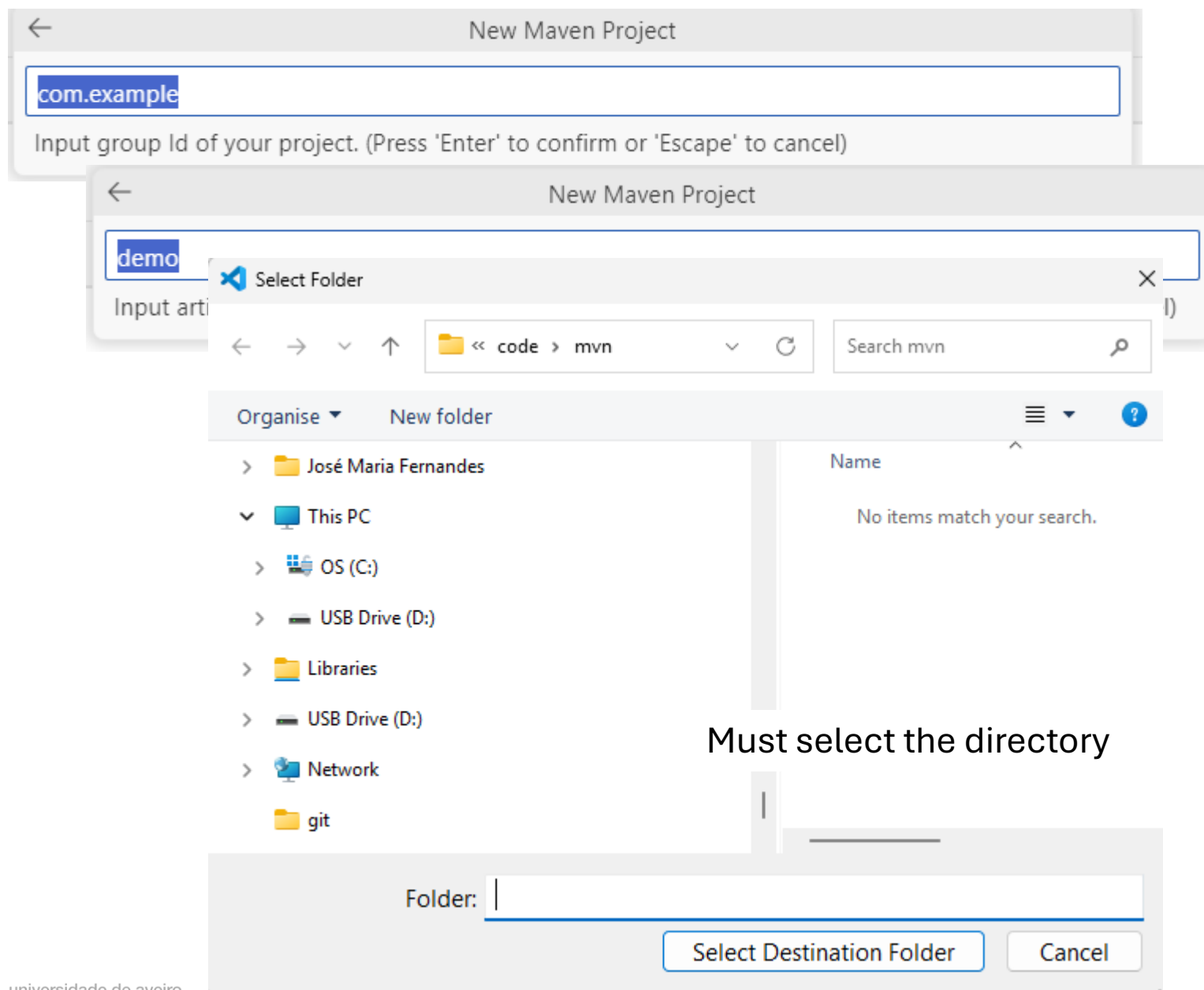


Create a project (VSCode)

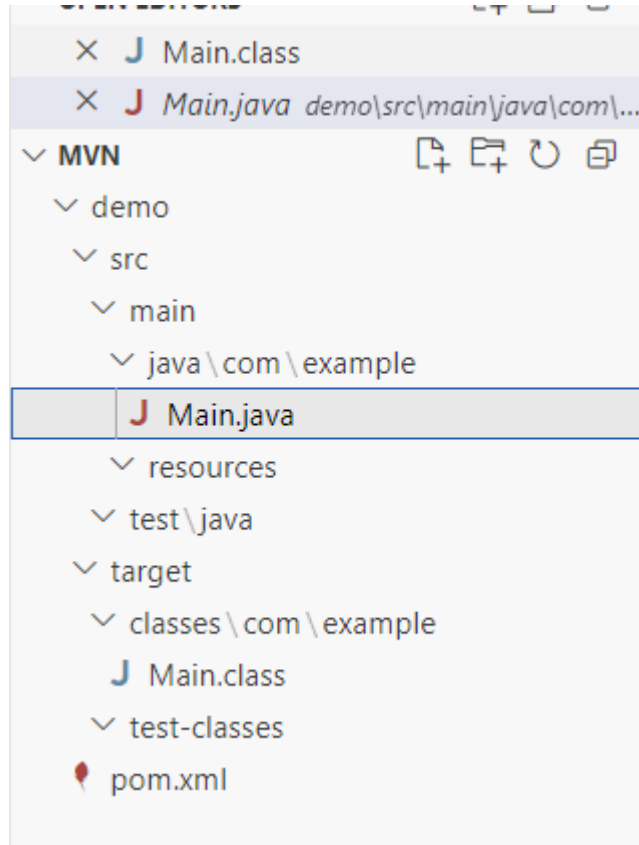


Create a project (VSCode)





The result



Maven creates the structure – this is the common structure of a shared by java archetypes

```
my-app
|-- pom.xml
`-- src
    |-- main
    |   |-- java
    |       |-- com
    |           |-- mycompany
    |               |-- app
    |                   |-- App.java
    |-- test
    |   |-- java
    |       |-- com
    |           |-- mycompany
    |               |-- app
    |                   |-- AppTest.java
```

We need JACKSON to handle json

The screenshot shows the Maven Central repository page for the **jackson-core** artifact. The page is titled "jackson-core" and shows the version "2.19.0". It indicates that the artifact is "Used in: 1566 components". The page has tabs for "Overview", "Versions", "Dependents", and "Dependencies". The "Overview" tab is selected, showing a "Description" and "Snippets". The description states: "Core Jackson processing abstractions (aka Streaming API), implementation for JSON". The "Snippets" section shows a code snippet for an Apache Maven dependency. A "Copy to clipboard" button is visible next to the snippet. The page also features a search bar and a "Feedback" button.

maven central repository

maven central repository Search

API

jackson-core 2.19.0

Used in: 1566 components

Overview Versions Dependents Dependencies

Overview

Description

Core Jackson processing abstractions (aka Streaming API), implementation for JSON

Snippets Apache Maven Copy to clipboard

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-core</artifactId>
  <version>2.19.0</version>
</dependency>
```

Feedback

EXPLORER

OPEN EDITORS

- Main.class
- pom.xml demo**

MVN

- demo
 - src
 - main
 - java\com\example
 - Main.java**
 - resources
 - test\java
 - target
 - classes\com\example
 - Main.class
 - test-classes
 - pom.xml**

OUTLINE

MAVEN

- demo com.example:demo
 - > Lifecycle
 - > Plugins
 - > Dependencies
 - > Favorites
 - > Profiles

Search bar: jackson-core

Input keywords to search artifacts from Maven Central Repository. (Press 'Enter' to confirm or 'Escape' to cancel)

URL: <http://maven.apache.org/POM/4.0.0/>

File Edit Selection View Go Run Terminal Help

mvn

EXPLORER

OPEN EDITORS

- Main.class
- pom.xml demo

MVN

- demo
 - src
 - main
 - java\com\example
 - Main.java
 - resources
 - test\java
 - target
 - classes
 - generated-sources
 - generated-test-sources
 - maven-archiver
 - maven-status
 - test-classes
 - demo-1.0-SNAPSHOT.jar
 - pom.xml

OUTLINE

MAVEN

- demo com.example:demo
 - Lifecycle
 - Plugins
 - Dependencies
 - com.fasterxml.jackson.core:jack...
 - Favorites
 - Profiles

Main.class pom.xml

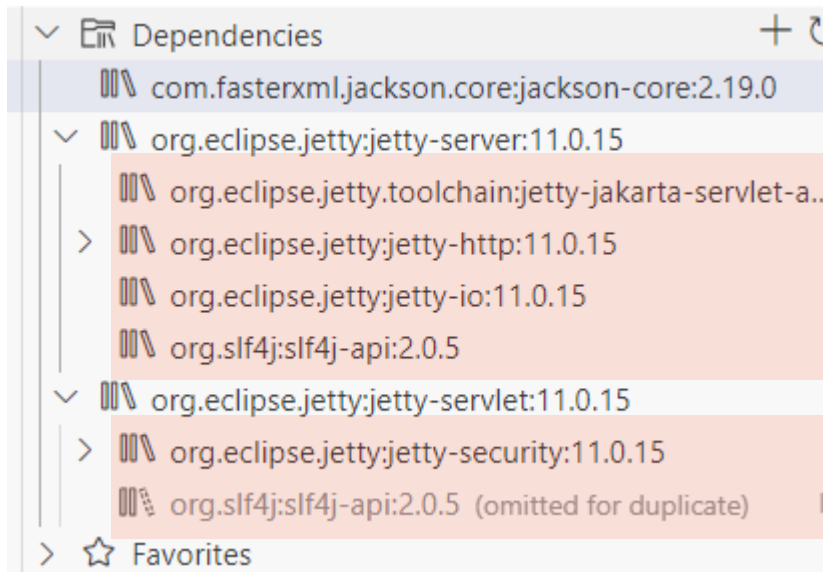
```
demo > pom.xml
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
10
11 <properties>
12 |   <maven.compiler.source>17</maven.compiler.source>
13 |   <maven.compiler.target>17</maven.compiler.target>
14 </properties>
15
16 <dependencies>
17 |   <dependency>
18 |     <groupId>com.fasterxml.jackson.core</groupId>
19 |     <artifactId>jackson-core</artifactId>
20 |     <version>2.19.0</version>
21 |   </dependency>
22 </dependencies>
23
24
25 <build>
26 |   <plugins>
27 |     <plugin>
28 |       <groupId>org.apache.maven.plugins</groupId>
29 |       <artifactId>maven-compiler-plugin</artifactId>
```

PROBLEMS 1

1 pom.xml demo

The build file has been changed and may need reload to make it effective. Java(0) [Ln 1, Col 1]

Maven identifies (in)direct dependencies



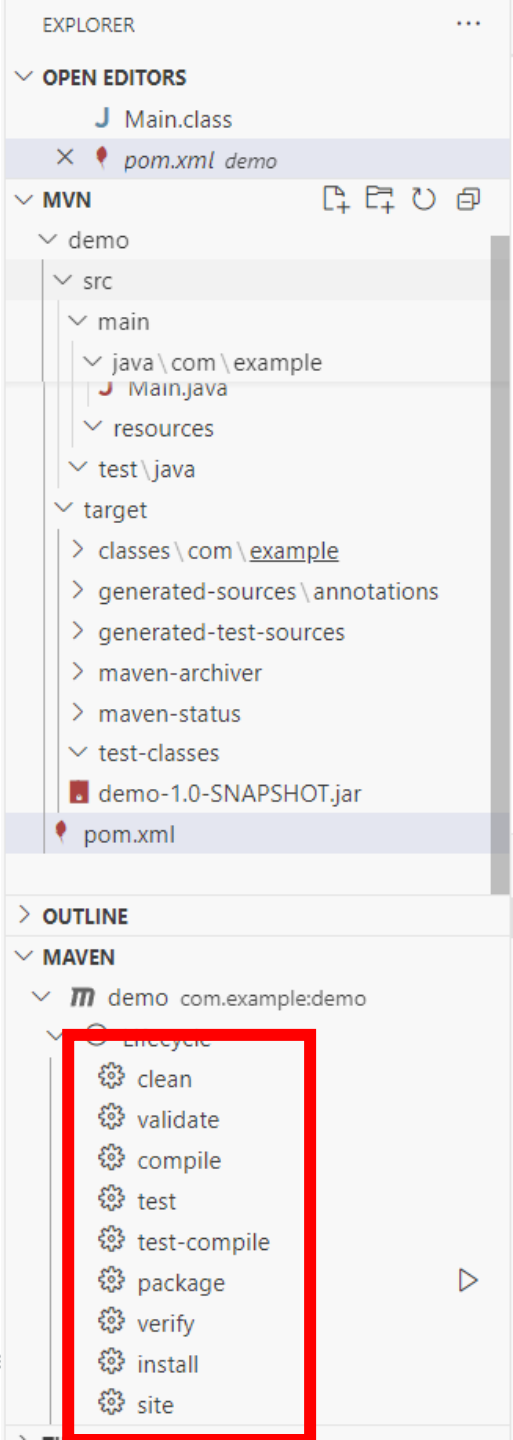
```
<dependencies>
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-core</artifactId>
    <version>2.19.0</version>
  </dependency>
  <dependency>
    <groupId>org.eclipse.jetty</groupId>
    <artifactId>jetty-server</artifactId>
    <version>11.0.15</version>
  </dependency>
  <dependency>
    <groupId>org.eclipse.jetty</groupId>
    <artifactId>jetty-servlet</artifactId>
    <version>11.0.15</version>
  </dependency>
</dependencies>
```

We are skipping details on...

- Lifecycles
 - Not critical for basic java projects (as in POO)
 - Useful for enterprise level
- Plugins
 - Automation
 - Adapt to context
- Details the maven command line interface

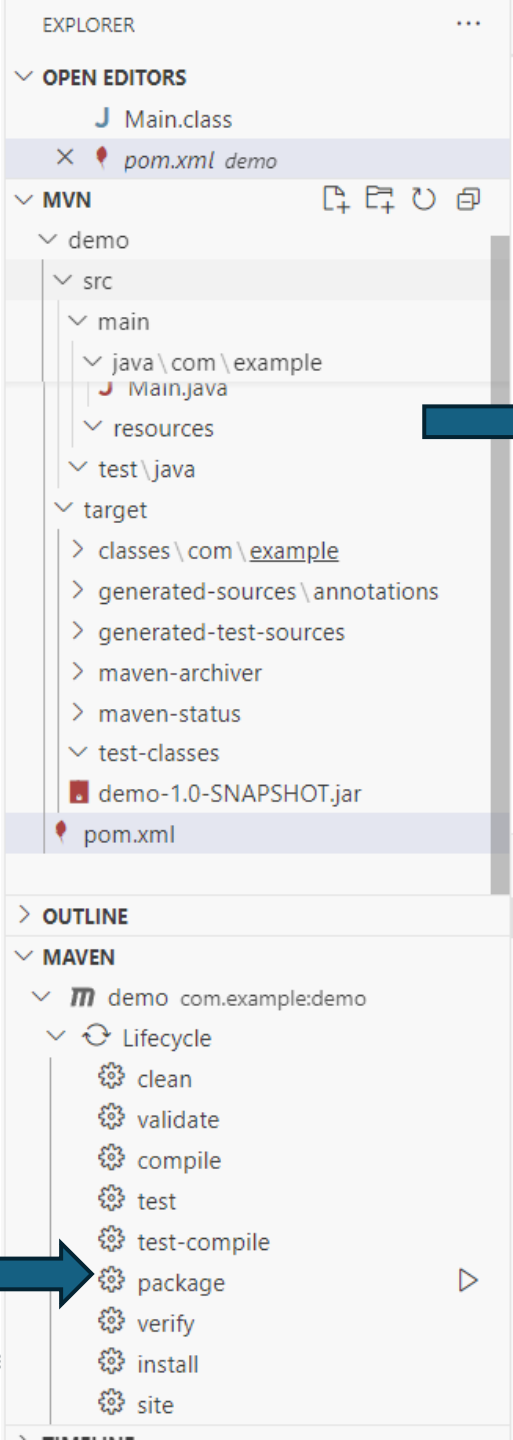
Short version

- Lifecycle
 - Stages / operations that maven execute
 - When available uses our configuration or plugins
- Plugins
 - Maven extensions that perform changes on project
 - Create files, change code, ...
 - Usually used in specific lifecycle stages
- What you need to know
 - Find the plugin that does what you need
 - Get the plugin configuration
 - Cut and paste into pom.xml
 - ... some magic
 - You have the result



lifecycle: the usefull

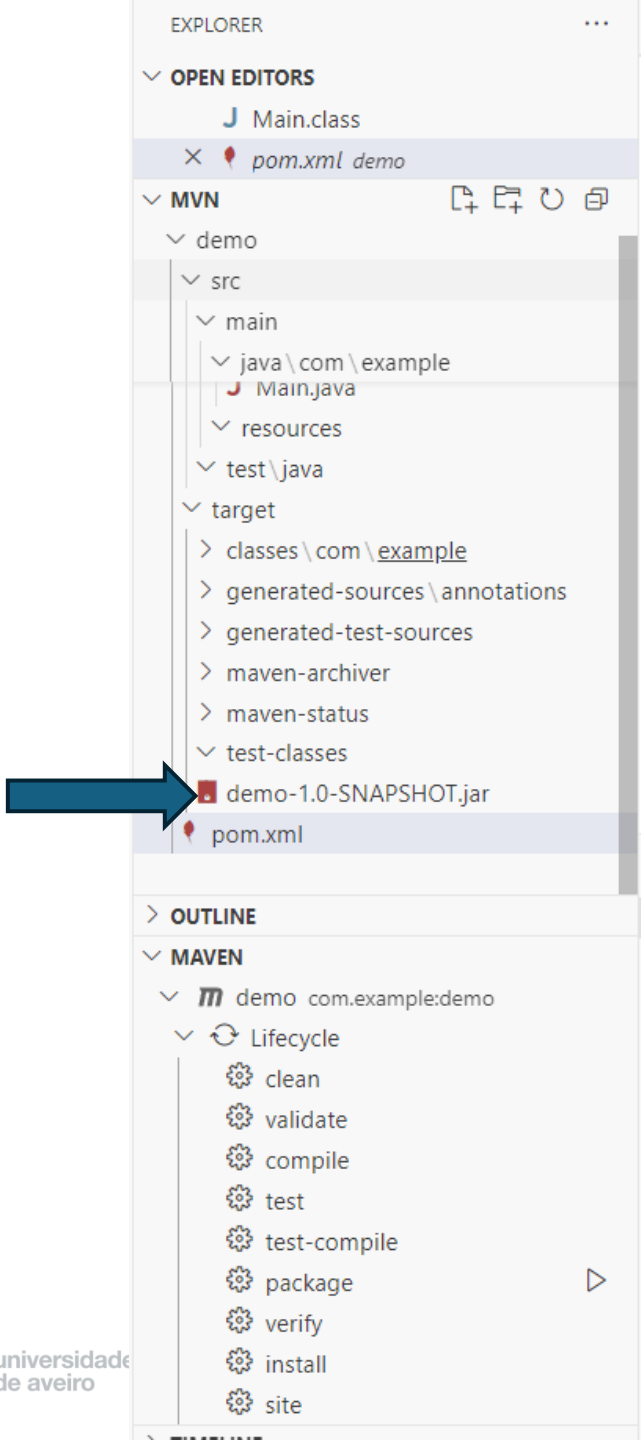
- clean – deletes the target folder.
- compile – turns source files into class files.
- ~~• test – runs a project's unit tests.~~
- package – creates an artifact such as a JAR, ZIP or WAR file.
- ~~• install – moves a created artifact into a Maven repository.~~
- ~~• And much more ...~~



creating you package

`mvn package -f "c:\code\mvn\demo\pom.xml"`

```
mvn package -f "c:\code\mvn\demo\pom.xml"
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:demo >-----
[INFO] Building demo 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ demo ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ demo ---
[INFO] Changes detected - recompiling the module! :input tree
[WARNING] File encoding has not been set, using platform encoding Cp1252, i.e. build is platform dependent!
[INFO] Compiling 1 source file with javac [debug target 17] to target\classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ demo ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory c:\code\mvn\demo\src\test\resources
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ demo ---
[INFO] Changes detected - recompiling the module! :dependency
[WARNING] File encoding has not been set, using platform encoding Cp1252, i.e. build is platform dependent!
[INFO]
[INFO] --- surefire:3.1.2:test (default-test) @ demo ---
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ demo ---
[INFO] Building jar: c:\code\mvn\demo\target\demo-1.0-SNAPSHOT.jar
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 4.719 s
[INFO] Finished at: 2025-05-06T14:33:45+01:00
[INFO]
PS C:\code\mvn>
```



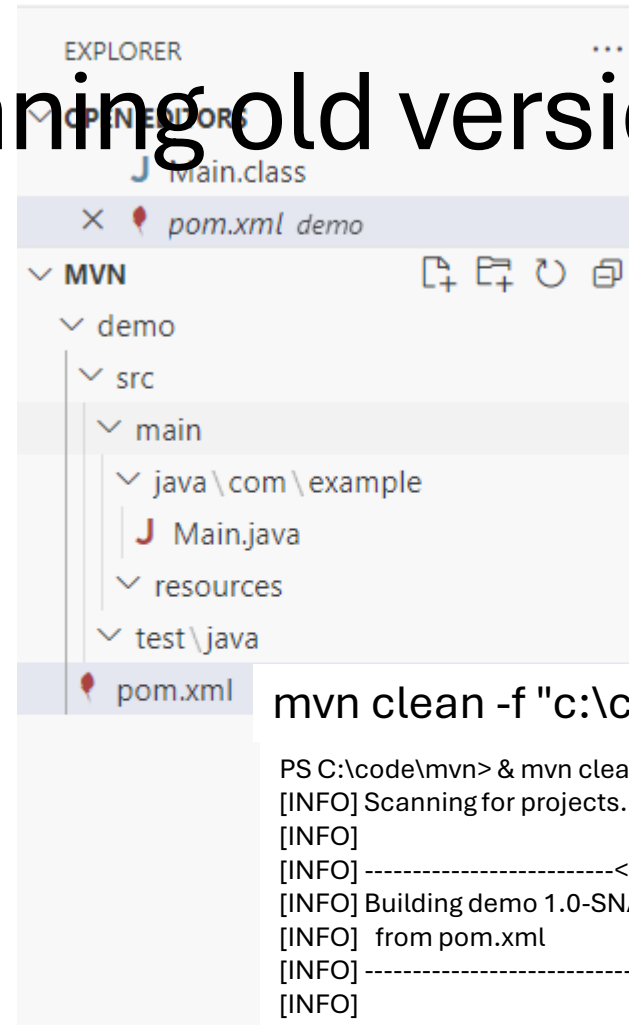
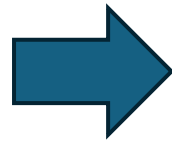
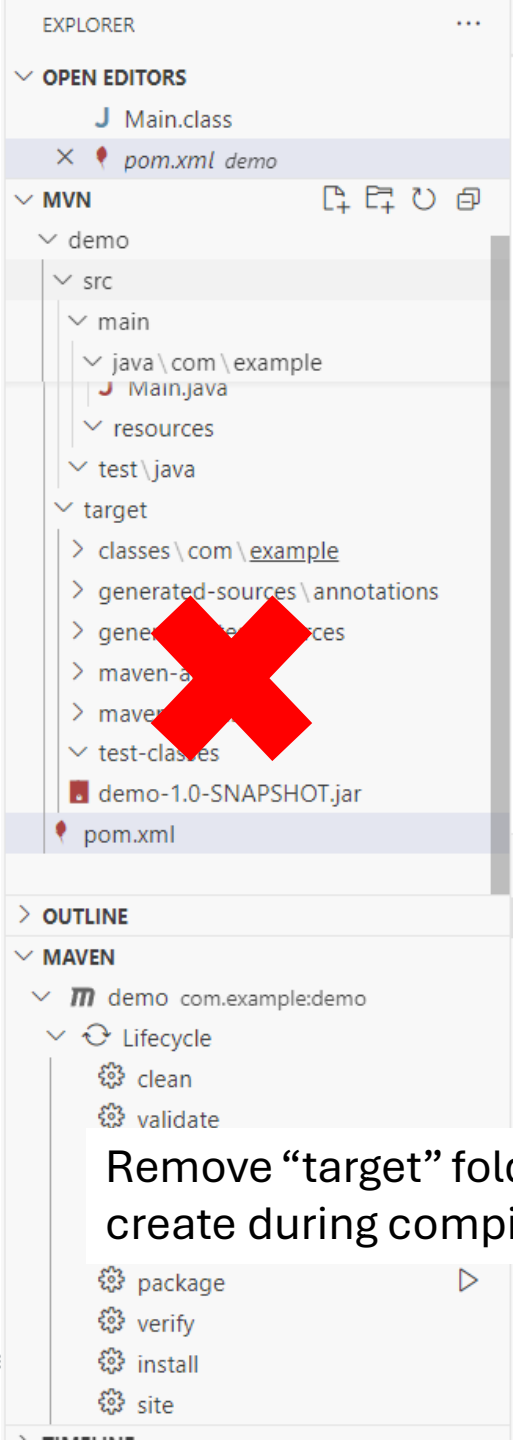
creating you package

```
mvn package -f "c:\code\mvn\demo\pom.xml"
```

Create a jar file containing the compiled class (to include dependencies may need to use a plugin)

```
mvn package -f "c:\code\mvn\demo\pom.xml"
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:demo >-----
[INFO] Building demo 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @
[WARNING] Using platform encoding (Cp1252 actually) to copy
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ demo ---
[INFO] Changes detected - recompiling the module! :input tree
[WARNING] File encoding has not been set, using platform encoding Cp1252, i.e. build is platform dependent!
[INFO] Compiling 1 source file with javac [debug target 17] to target/classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ demo ---
[WARNING] Using platform encoding (Cp1252 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory c:\code\mvn\demo\src\test\resources
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ demo ---
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ demo ---
[INFO] Building jar: c:\code\mvn\demo\target\demo-1.0-SNAPSHOT.jar
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 4.719 s
[INFO] Finished at: 2025-05-06T14:33:45+01:00
[INFO]
PS C:\code\mvn>
```

Cleaning old versions and “junk”



```
mvn clean -f "c:\code\mvn\demo\pom.xml"
```

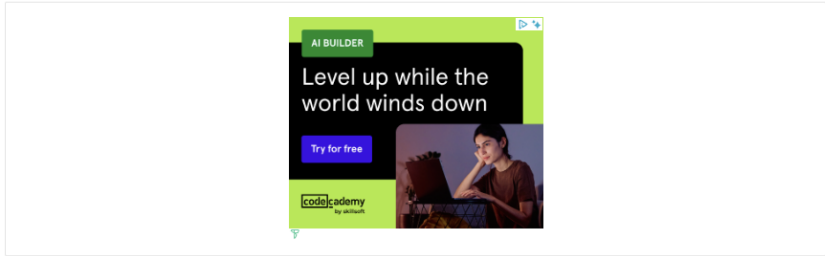
```
PS C:\code\mvn> & mvn clean -f "c:\code\mvn\demo\pom.xml"
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:demo >-----
[INFO] Building demo 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ demo ---
[INFO] Deleting c:\code\mvn\demo\target
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.384 s
[INFO] Finished at: 2025-05-06T14:40:02+01:00
[INFO]
PS C:\code\mvn>
```

Remove “target” folder i.e. remove all files
create during compilation and deployment

I need to deploy a jar with my program

- Deliver my program (classes not java code) in a jar
 - My classes
 - All dependencies i.e. jars needed

How to Create an Executable JAR with Maven



Last updated: August 7, 2025

Written by:
baeldung

Reviewed by:
Grzegorz Piwowarek

Maven

Maven Plugin

Code your way through and build up a solid, practical foundation of Java:
[>> Learn Java Basics](#)

1. Overview

In this quick tutorial, we'll focus on **packaging a Maven project into an executable Jar file**.
When creating a *jar* file, we usually want to run it easily, without using the IDE. To that end, we'll discuss the configuration and pros/cons of using each of these approaches for creating the executable.

Further reading:

Apache Maven Tutorial
A quick and practical guide to building and managing Java projects using Apache Maven.
Read more →

Where Is the Maven Local Repository?
Quick tutorial showing you where Maven stores its local repo and how to change that.
Read more →

Spring with Maven BOM
Learn how to use a BOM, Bill of Materials, in your Spring Maven project.
Read more →

2. Configuration

We don't r and have i <https://www.baeldung.com/executable-jar-with-maven>

In our example, we created a Java class named *ExecutableMavenJar*.

We also need to make sure that our *pom.xml* contains these elements:

Fat Jar Files

- Uber Jars or executable Jars,
- take the concept of Jar files a step further by
- include **all dependencies within the archive.**
- self-contained approach
 - simplifies deployment,
 - as the application can be run **without relying on external libraries.**

Creating a Fat Jar File:

Using tools like Maven or Gradle, you can generate a Fat Jar easily.

Use proper plugin in pom.xml

If you're using Maven, use the `maven-shade-plugin` in your `pom.xml` :

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-shade-plugin</artifactId>
      <version>3.5.0</version>
      <executions>
        <execution>
          <phase>package</phase>
          <goals><goal>shade</goal></goals>
          <configuration>
            <transformers>
              <transformer
                implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">

                <mainClass>EmbeddedJettyExample</mainClass> <!-- Replace with your full
class name -->

              </transformer>
            </transformers>
          </configuration>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

<https://maven.apache.org/plugins/maven-shade-plugin/>
<https://medium.com/@lavneesh.chandna/unveiling-the-maven-shade-plugin-a-comprehensive-guide-e878966f6ee8>

Create the jar

```
bash
```

[Copy](#)[Edit](#)

```
mvn clean package
```

This will produce a **fat JAR** (e.g., `your-artifact-name-jar-with-dependencies.jar`) in the `target/` directory.

run

2. Run the JAR

```
bash
```

[Copy](#)[Edit](#)

```
java -jar target/your-artifact-name-jar-with-dependencies.jar
```

Replace the file name with the actual one Maven generates.

It will run the class plugin field...

```
<mainClass>EmbeddedJettyExample</mainClass><!-- Replace with  
your full class name -->
```

and if I want to run a specific class?

and if I want to run a specific class within the jar ?

Great question! If you want to run a **specific class within a JAR**, not necessarily the one defined as the **Main-Class** in the manifest, you can do it like this:

✓ Command to Run a Specific Class from a JAR

bash

Copy

Edit

```
java -cp your-jar-file.jar your.package.ClassName
```



- **-cp** (or **-classpath**) tells Java to use your JAR for classes.
- Replace **your-jar-file.jar** with the name of your JAR.
- Replace **your.package.ClassName** with the **fully qualified name** of the class you want to run.



and if I want to run a specific class?

and if I want to run a specific class within the jar ?

Great question! If you want to run a **specific class within a JAR**, not necessarily the one defined as the **Main-Class** in the manifest, you can do it like this:

✓ Command to Run a Specific Class from a JAR

bash

Copy

Edit

```
java -cp your-jar-file.jar your.package.ClassName
```

- **-cp** (or **-classpath**) tells Java to use your JAR for classes.
- Replace **your-jar-file.jar** with the name of your JAR.
- Replace **your.package.ClassName** with the **fully qualified name** of the class you want to run.

If your jar does not have all dependencies...

If the class requires external dependencies (like Jetty), and your JAR is **not a fat JAR**, then you'll need to add those dependencies to the classpath too, like this:

```
java -cp "myserver.jar:lib/*" com.example.EmbeddedJettyExample
```

The jar with your code ...

```
'myserver.jar':
```

directory(ies) containing all
dependencies that are OUTSIDE the jar

```
lib/*
```

If your jar does not have all dependencies...

If the class requires external dependencies (like Jetty), and your JAR is **not a fat JAR**, then you'll need to add those dependencies to the classpath too, like this:

```
java -cp "myserver.jar:lib/*" com.example.EmbeddedJettyExample
```

On Windows, use `;` instead of `::`:

```
bash
```

```
java -cp "myserver.jar;lib/*" com.example.EmbeddedJettyExample
```

If your jar does not have all dependencies...

If the class requires external dependencies (like Jetty), and your JAR is **not a fat JAR**, then you'll need to add those dependencies to the classpath too, like this:

```
java -cp "myserver.jar:lib/*" com.example.EmbeddedJettyExample
```

The jar with your code ...

`myserver.jar`

directory(ies) containing all dependencies that are OUTSIDE the jar

`lib/*`

Th

How to Create an Executable JAR with Maven

1. Overview

In this quick tutorial, we'll focus on **packaging a Maven project into an executable Jar file**.

When creating a *jar* file, we usually want to run it easily, without using the IDE. To that end, we'll discuss the configuration and pros/cons of using each of these approaches for creating the executable.

How to Create an Executable JAR with Maven

The most important aspect here is the type — to create an executable *jar*, double-check the configuration uses a *jar* type.

- Now we can start using the various solutions.

```
<modelVersion>4.0.0</modelVersion>  
<groupId>com.baeldung</groupId>  
<artifactId>core-java</artifactId>  
<version>0.1.0-SNAPSHOT</version>  
<packaging>jar</packaging>
```

Goal – copy dependencies

- specify the *goal copy-dependencies*,
- copy dependencies into the specified *outputDirectory* i.e. a folder *libs* inside the project build directory (which is usually the *target* folder).

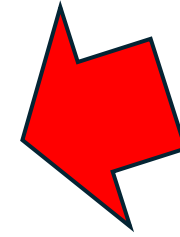
```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-dependency-plugin</artifactId>
  <executions>
    <execution>
      <id>copy-dependencies</id>
      <phase>prepare-package</phase>
      <goals>
        <goal>copy-dependencies</goal>
      </goals>
      <configuration>
        <outputDirectory>
          ${project.build.directory}/libs
        </outputDirectory>
      </configuration>
    </execution>
  </executions>
</plugin>
```

Maven jar plugin

- the *manifest* configuration. We add a classpath, with all dependencies (folder *libs/*), and provide the information about the main class.
- need to provide a fully qualified name of the class, which means it will include the package name.

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-jar-plugin</artifactId>
  <configuration>
    <archive>
      <manifest>
        <addClasspath>true</addClasspath>
        <classpathPrefix>libs/</classpathPrefix>
        <mainClass>
          com.baeldung.executable.ExecutableMavenJar
        </mainClass>
      </manifest>
    </archive>
  </configuration>
</plugin>
```

Apache Maven Shade Plugin



There are three main parts to this configuration.

First, `<shadedArtifactAttached>` marks all dependencies to be packaged into the *jar*.

Second, we need to specify [the transformer implementation](#); we used the standard one in our example.

Finally, we need to specify the main class of our application.

The output file will be named *core-java-0.1.0-SNAPSHOT-shaded.jar*, where *core-java* is our project name followed by snapshot version and plugin name.

- **pros** – dependencies inside the *jar* file, advanced control of packaging our artifact, with shading and class relocation
- **cons** – complex configuration (especially if we want to use advanced features)

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-shade-plugin</artifactId>
  <executions>
    <execution>
      <goals>
        <goal>shade</goal>
      </goals>
      <configuration>
        <shadedArtifactAttached>true</shadedArtifactAttached>
        <transformers>
          <transformer implementation=
            "org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
            <mainClass>com.baeldung.executable.ExecutableMavenJar</mainClass>
          </transformer>
        </transformers>
      </configuration>
    </execution>
  </executions>
</plugin>
```

What was addressed

- Maven (focus in java)
 - pom.xml
 - dependencies and plugins
 - adding dependencies
- deploy a java application
 - jar and fat jar
 - the maven pluggins to deploy

Some links on maven

- Apache Maven Series
 - <https://www.baeldung.com/maven-series>
- Getting Started With Maven - A Beginner's Guide to Efficient Java Build Management
 - <https://hackernoon.com/getting-started-with-maven-a-beginners-guide-to-efficient-java-build-management>
- Maven in 5 Minutes
 - <https://maven.apache.org/guides/getting-started/maven-in-five-minutes.html>
- Maven Getting Started Guide
 - <https://maven.apache.org/guides/getting-started/>

The END